Groq Tensor Streaming Processor 架构综述：软件定义硬件与确定性计算的范式革命

Claude (Anthropic)

2026 年 4 月

摘要

Groq Tensor Streaming Processor (TSP) 代表了一种根本不同的 AI 加速器设计哲学。本文基于 ISCA 2020/2022 两篇奠基性论文及 6 篇技术分析文章，系统综述了 Groq TSP 的架构创新。TSP 采用功能切片微架构，将 220MB SRAM 与矩阵/向量运算单元在空间上交错排列；通过软件定义硬件方法，将传统硬件中的动态调度、缓存管理、网络路由等复杂逻辑完全移至编译器静态处理；以确定性执行为核心，消除了缓存、仲裁器、分支预测器等所有非确定性元件，实现了从单芯片到万卡集群的完全可预测性能。

本文深入分析了 TSP 的 144 路 VLIW 指令集、320 元素向量处理单元、320×320 MACC 矩阵运算阵列、软件调度网络以及基于硬件对齐计数器 (HAC) 和软件对齐计数器 (SAC) 的多芯片同步机制。同时探讨了 Groq 被 NVIDIA 以 200 亿美元收购的技术背景，分析了 DSA 与 GPGPU 架构的融合趋势，并对 AI 加速器架构的未来演进提出了批判性思考。

关键词： Groq TSP；软件定义硬件；确定性执行；功能切片；张量流处理；AI 加速器；多芯片同步

目录

1 引言 **2**

1.1 背景与动机 2

1.2 核心创新点 2

1.3 论文组织 2

2 TSP 微架构设计 **3**

2.1 功能切片：重新组织处理器微架构 3

2.2 指令集架构：144 路 VLIW 6

2.3 存储层次：扁平化 SRAM 设计 7

2.4 各功能单元深入分析 9
2.4.1 指令控制单元 (ICU) 与指令流调度 9
2.4.2 矩阵运算单元 (MXM) 10
2.4.3 向量运算单元 (VXM) 与逐元素操作 10
2.4.4 数据重排单元 (SXM) 11
2.4.5 芯片间通信单元 (C2C) 与全局扩展 11
2.4.6 流寄存器机制与时空确定性 11

3 软件定义硬件与编译器架构 12
3.1 静态-动态接口划分 12
3.2 Groq 编译器架构 12
3.3 编译器优化技术详解 13
3.3.1 周期精确指令调度 13
3.3.2 模调度与软件流水线 15
3.3.3 寄存器压力感知调度 15
3.3.4 内存分配与数据布局优化 15
3.3.5 两阶段调度策略 15
3.4 运行时系统 16

4 多芯片同步与软件调度网络 16
4.1 从单芯片到多芯片的确定性扩展 16
4.2 软件调度网络 (SSN) 18

5 与 GPGPU 架构的对比分析 21
5.1 计算密度与能效 21
5.2 延迟与可预测性 23
5.3 编程模型与生态 23

6 NVIDIA 收购与技术整合展望 24
6.1 200 亿美元收购的技术逻辑 24
6.2 从 TSP 到 Groq 3 LPX：架构演进与产业定位的收敛 24

7 结论与批判性思考 27
7.1 主要贡献总结 27
7.2 局限性与批评 27
7.3 未来研究方向 28
7.4 最终思考 28

1 引言

1.1 背景与动机

深度学习工作负载的爆炸式增长推动了专用加速器 (Domain-Specific Architecture, DSA) 的蓬勃发展。从 Google 的 TPU 到 NVIDIA 的 Tensor Core，业界一直在探索如何以更高效率执行神经网络计算。然而，这些设计大多遵循 GPGPU 的" 硬件中心" 范式：复杂的硬件调度逻辑、多层缓存层次结构、动态分支预测——这些机制虽然提高了平均性能，但也引入了不可预测的执行延迟和显著的面积/功耗开销。

Groq 公司于 2020 年在 ISCA 发表的 TSP 架构论文 [1] 提出了一个根本性的质疑：如果编译器能够在编译时精确知道程序的执行路径，为什么还要在硬件中保留复杂的动态调度逻辑？基于这一洞察，TSP 架构采用了一种" 软件定义硬件" 的设计理念——将控制和调度操作完全交由软件 (编译器) 完成，硬件仅负责执行。

这一设计理念的物理实现是一枚 14nm 工艺、25 × 29 mm、包含 268 亿晶体管的 ASIC 芯片 [1]。其核心规格令人印象深刻：220 MB 片上 SRAM 提供 55 TB/s 的 SRAM 带宽和 20 TB/s 的流寄存器带宽；4 个独立的 320 × 320 MACC 阵列提供 820 TeraOps/s 的峰值性能 (在 900MHz 频率下)；5,120 个 32 位向量 ALU 支持 FP16 和 INT8 原生数据类型；144 路 VLIW 指令发射支持多达 10,440 芯片的全局同步网络。

1.2 核心创新点

TSP 架构的革命性创新可以从三个层面理解。在芯片层面，TSP 采用功能切片架构，将内存 (MEM)、向量运算 (VXM)、矩阵运算 (MXM)、数据重排 (SXM) 四种功能单元在空间上交错排列，通过流寄存器实现确定性数据传输 [1]。这种设计消除了缓存、仲裁器、乱序执行等传统非确定性元件，使编译器能够进行周期精确的指令调度。

在编译器层面，TSP 将传统硬件控制逻辑 (指令调度、内存管理、数据流控制) 完全移至编译器。编译器精确知道每条指令的执行周期、每个数据的存储位置、每次通信的传输延迟。这种" 全知" 视角使编译器能够生成最优的指令序列，而硬件只需按序执行 [3]。

在系统层面，TSP 的多芯片扩展不依赖传统的动态路由网络，而是采用软件调度网络 (Software-Scheduled Networking, SSN)。所有网络路由决策在编译时确定，通过 Dragonfly 拓扑实现高带宽、低延迟的全局通信 [2]。硬件对齐计数器 (HAC) 和软件对齐计数器 (SAC) 机制确保数千个芯片能够保持纳秒级同步 [5]。

1.3 论文组织

本文剩余部分组织如下：第2节详细分析 TSP 的微架构设计，包括功能切片、指令集架构和存储层次；第3节探讨软件定义硬件的实现机制，重点分析编译器在指令调度和内存管理中的核心作用；第4节深入讨论多芯片同步机制，包括 HAC/SAC 和软件调

度网络；第5节将 TSP 与传统 GPGPU 架构进行对比分析；第6节探讨 Groq 被 NVIDIA 收购后的技术整合方向；第7节总结全文并提出批判性思考。

2 TSP 微架构设计

2.1 功能切片：重新组织处理器微架构

传统多核处理器采用”核心-缓存-互连”的层次化设计。每个核心包含私有 L1/L2 缓存，多个核心通过片上网络 (NoC) 共享 L3 缓存和主存。这种设计面临三个根本性问题：首先，深度学习工作负载的数据局部性难以被固定大小的缓存捕获，导致频繁的数据移动和缓存未命中；其次，缓存未命中、仲裁竞争、网络拥塞等因素导致执行时间难以预测；第三，乱序执行、推测执行、分支预测等机制消耗大量晶体管却仅带来边际性能提升。

Abts 等人 [1] 指出，在深度学习工作负载中，数据并行性极为丰富，可以天然映射到张量硬件。问题的关键不在于计算能力，而在于如何高效地将数据送入计算单元。基于这一观察，TSP 采用功能切片方法重新组织处理器微架构。图1展示了 TSP 的功能切片架构，芯片被划分为 20 个 SuperLane，每个 SuperLane 包含功能互补的四种单元。

图 1: TSP 功能切片架构图 [1]。图中显示了 MEM West/East、quad 组织方式、以及 MXM/VXM/SXM/C2C 等单元的空间布局。

表1总结了四种功能单元的职责分工：

TSP 的编程模型是**生产者-消费者流式模型**，区别于传统的 load-store 架构。在传统 RISC 架构中，执行向量加法 $Z = X + Y$ 需要四条指令：LOAD、LOAD、ADD、STORE，操作数需在通用寄存器和内存间多次搬运。TSP 中，功能切片以流式方式交互：MEM 切片读取数据到流寄存器，数据在流寄存器中”流动”经过 VXM 切片时触发计算，结果直接流到下游 MEM 切片写入内存。指令从 ICU 垂直 (Northward) 流入各功能切片，数据水平 (East/West) 流动，两者在时空上精确交叉——编译器确保在特定

表 1: TSP 功能单元分类

单元类型	缩写	功能描述
内存单元	MEM	管理片上 SRAM，提供每个时钟周期 320 字的读写带宽
向量运算单元	VXM	执行元素级运算 (激活函数、归一化、点积)，每 lane 16 个 ALU
矩阵运算单元	MXM	执行矩阵乘法和卷积，每个 SuperLane 包含一个 320×16 MACC 阵列
数据重排单元	SXM	执行数据转置、广播、切片等重排操作

周期，特定指令与特定数据在特定功能单元相遇。这种**时空确定性**是 TSP 实现高效执行的核心。

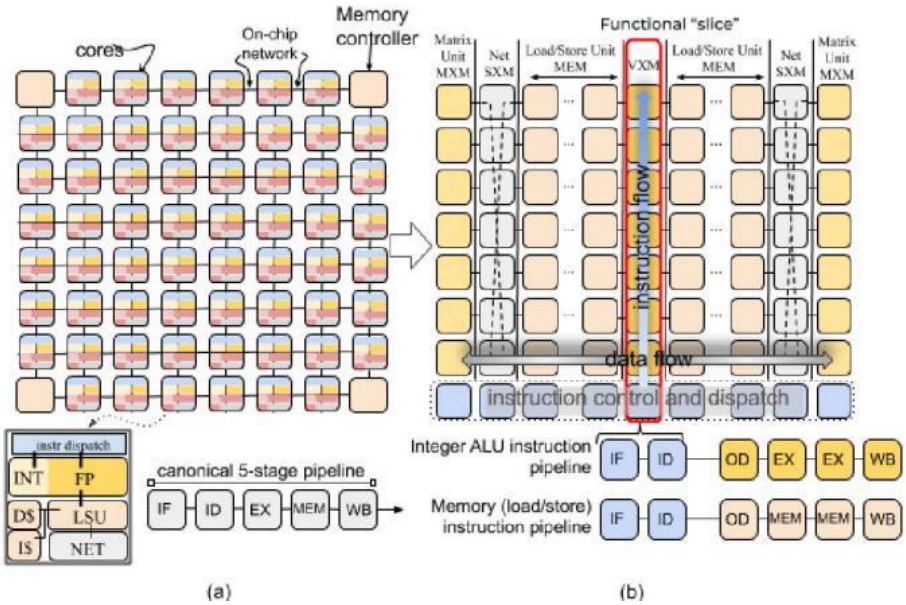

图 2: 传统多核架构 (a) 与 TSP 功能切片架构 (b) 的对比。功能切片将指令流垂直 (Northward) 分发，数据流水平 (East/West) 流动，实现时空确定性执行 [1]

图3展示了复合运算 $Y = \mathrm{ReLU}(W \times X + b)$ 在 TSP 上的空间融合执行示例：矩阵乘法在 MXM 完成，结果经 SXM 分发后进入 VXM 执行累加 (Accum)、偏置加法 (Add)、ReLU 激活，最终通过 MEM 写回。这种空间融合避免了中间结果写回 SRAM，显著降低了数据搬运开销。

图4展示了数据在这些功能单元间的流动方式。

每个 SuperLane 包含 16 个并行 lane，每个 lane 宽度为 320 元素。这种设计允许 TSP 以 SIMD(单指令多数据) 方式同时处理大量数据。关键创新在于流寄存器机制：流寄存器位于功能单元之间，作为生产者-消费者缓冲；数据从一个单元流出后立即被下一个单元消费，无需通过 SRAM 中转；编译器精确知道数据在流寄存器中的停留时间，可以进行精确调度。这种设计实现了极高的数据复用率。对于 ResNet50 推理，TSP 可以在 batch size 为 1 的情况下达到 20.4K images/second 的处理速度，延迟仅 49 微秒

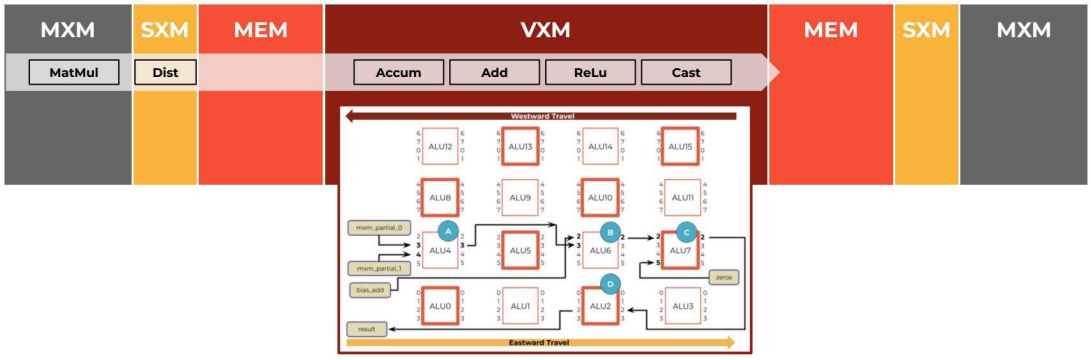

图 3: TSP 空间融合执行示例：复合运算 $Y = \mathrm{ReLU}(W \times X + b)$ 的硬件映射。矩阵乘法在 MXM 完成，结果经 SXM 分发后进入 VXM 执行累加、偏置加法、ReLU 激活，最终通过 MEM 写回 [1]

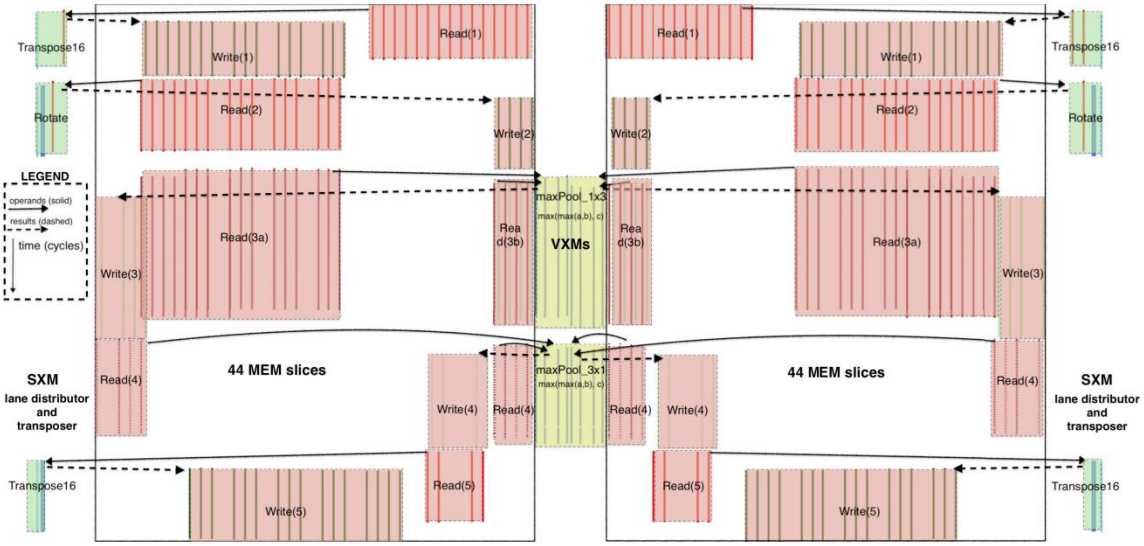

图 4: TSP 数据流与内存访问模式示意图 [1]。图中显示了 Read/Write 操作在 44 个 MEM slice 之间的流动，以及 VXM、SXM 等单元的交互。

[1]——这比同期 GPU 快 4 倍以上。

2.2 指令集架构：144 路 VLIW

TSP 采用 144 路 VLIW(甚长指令字) 指令集。每条 VLIW 指令同时控制 20 个 SuperLane 与 5 个功能切片共 100 个独立控制信号，以及额外的同步、分支、网络控制信号。这种极端的 VLIW 设计之所以可行，是因为编译器拥有完整的信息：所有指令的执行延迟是固定的 (确定性设计)，所有数据的存储位置是已知的 (无缓存)，所有依赖关系可以在编译时解析 (无乱序执行)。

TSP 指令集包含多种关键指令类型 [3]，如代码1所示。

```
// 1. 基本运算指令
NOP       // 空操作，用于时序对齐
FMA       // 乘加运算，MXM核心操作
VADD/VMUL// 向量加减乘除

// 2. 内存访问指令
LOAD/STORE      // SRAM读写
PREFETCH        // 数据预取（软件控制）
GATHER/SCATTER // 非连续内存访问

// 3. 同步指令
SYNC          // 屏障同步
NOTIFY        // 事件通知
RUNTIME_DESKEW// 运行时去歪斜

// 4. 网络指令
SEND/RECV     // 芯片间数据发送/接收
MULTICAST     // 多播通信
REDUCE        // 归约操作

// 5. 控制指令
IFETCH        // 指令获取
BRANCH        // 条件分支（无推测）
SWITCH        // 上下文切换
```

Listing 1: TSP 核心指令类型

值得注意的是，TSP 的分支指令**不进行推测执行**。编译器在分支指令后插入足够多的 NOP，确保分支结果确定后才执行后续指令。这牺牲了部分性能换取了完全的可预测性。

2.3 存储层次：扁平化 SRAM 设计

TSP 最激进的设计决策之一是**完全摒弃片外 DRAM**。整个 220 MB 存储由片上 SRAM 构成，采用层次化的组织方式实现高并发访问。

物理组织结构。SRAM 存储在物理上分为 West Hemisphere 和 East Hemisphere 两个半区，对应芯片的左右两侧。如图5所示，每个 MEM slice 在物理上是一条**纵向的 SRAM 条带**，由 20 个 tiles 沿南北方向垂直堆叠而成，贯穿全部 20 个 SuperLane。每侧 Hemisphere 在水平方向上并排排列 44 条独立的 MEM slice，全片共 88 条。slice MEM_0 最靠近 VXM，MEM_{43} 最靠近 SXM。单个 slice 容量为 2.5 MiB，总计 $88 \times 2.5 = 220$ MiB。

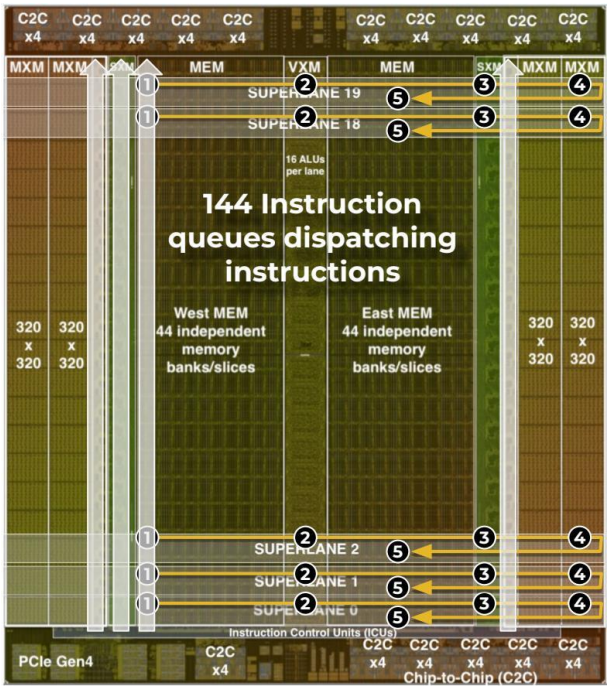

图 5: TSP 芯片物理版图与 MEM slice 排布。芯片两侧对称分布 West/East 各 44 条纵向 MEM slice（全片共 88 条），每个 slice 由 20 个 tiles 垂直堆叠；芯片中央为 VXM 向量单元，左右两侧边缘各部署 2 个 320×320 MACC 阵列 (MXM)，20 个 SuperLane 纵向贯穿整个芯片。图片来源：Abts et al., ISCA 2020[1]

图6展示了 TSP 芯片的地址编码方案。

Bank 划分与数据布局。每个 slice 内部进一步划分为两个独立的 bank：**Bank 0** 专门用于存储**模型权重**，**Bank 1** 专门用于存储**激活值**。这种分离允许权重和激活值在硬件层面并行访问——当 MXM 从 Bank 0 读取权重进行矩阵运算时，VXM 可以同时从 Bank 1 读取激活值执行逐元素操作。

SRAM 带宽计算。每个 bank 在每个时钟周期可以提供 320 字节（对应 320 lane）的读写带宽。全片 SRAM 总带宽计算公式为：

$$M = 2 \times 44 \times 2 \times 320 \times f \text{ (bytes/s)}$$

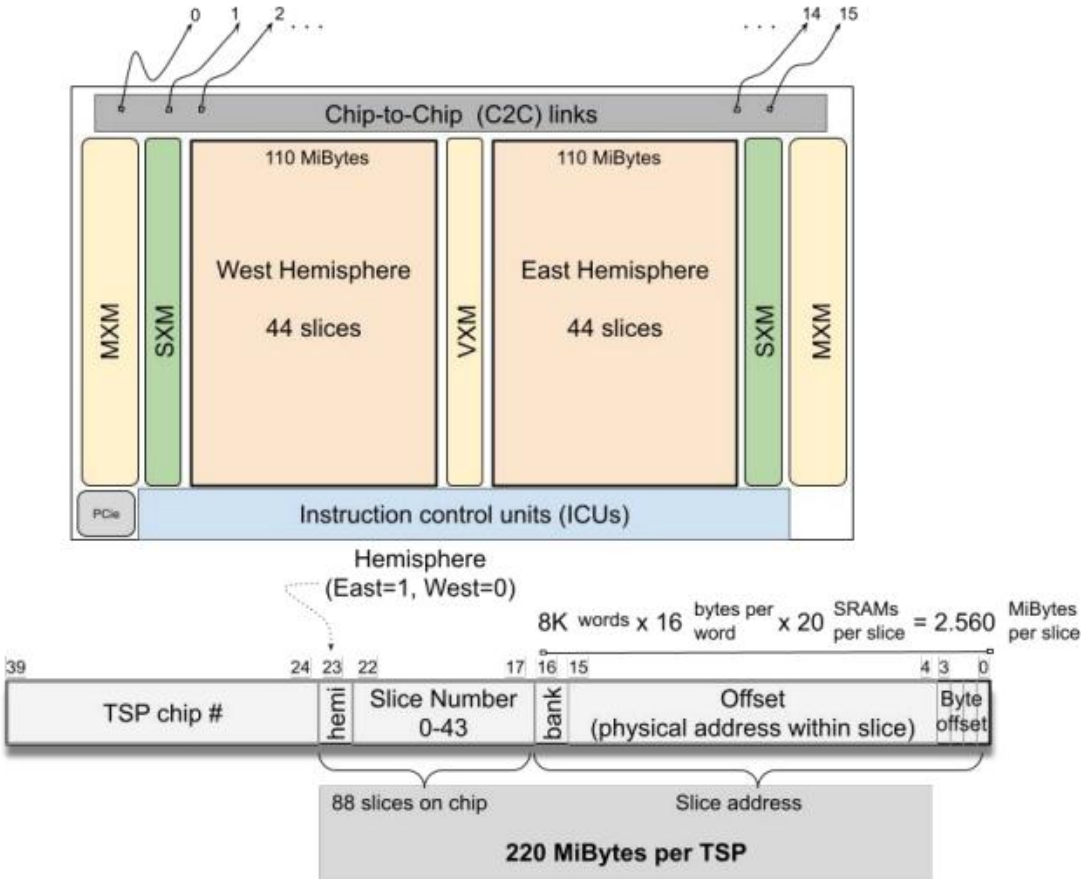

图 6: TSP 芯片物理布局与地址编码方案 [2]。West/East Hemisphere 各 44 slices，共 220 MiBytes，以及地址编码的详细格式。

其中 f 为时钟频率。TSP 实际工作频率为 900 MHz，但论文中假设 1 GHz 进行理论分析。在 1 GHz 主频下，SRAM 总带宽为：

$$M = 2\ \text{hem} \times 44\frac{\text{slices}}{\text{hem}} \times 2\frac{\text{banks}}{\text{slice}} \times 320\frac{\text{bytes}}{\text{cycle}} \times 1\ \text{GHz} = 55\ \text{TB/s}$$

向量布局与地址映射。TSP 的 SRAM 以 16 字节为最小寻址字，每个字的 16 个字节按 lane index 并行分布于同一个 SuperLane 的 16 个 lane 上（byte 0 在 lane 0，byte 1 在 lane 1，以此类推），从而保证一个 16 字节向量可被单个 SuperLane 在同一周期并行读取。

PGAS 存储模型与数据流向。编译器采用 **PGAS**（Partitioned Global Address Space，分区全局地址空间）模型管理片上 SRAM：每个张量被显式分配到特定的 MEM slice 范围和 bank，程序可以全局引用这些地址，但数据在物理上被静态分片到不同 slice，不存在缓存迁移或隐式数据移动。读写指令除携带 SRAM 地址外，还需同时指定数据流向——**Eastward**（向东，即向芯片右侧的 Hemisphere 流动）或 **Westward**（向西，向左侧的 Hemisphere 流动）。由于存储访问延迟固定且无缓存未命中，编译器能够精确计算每个内存操作的完成周期，从而将数据搬运与后续计算精确流水线化。相比之下，GDDR6 显存的典型带宽仅为 1-2 TB/s。

由于没有缓存，TSP 要求编译器进行显式内存管理。程序员或编译器需要明确指定每个张量的 SRAM 存储位置、数据的生命周期和替换策略、内存 bank 之间的数据移动。这种显式管理增加了编译复杂度，但消除了缓存未命中的不可预测性。对于 ResNet50，TSP 编译器可以精确安排每层的权重加载和激活计算，实现零气泡流水线 [1]。

Groq 3 LPU 在存储子系统上进行了重大升级 [4]：SRAM 容量从 220 MB 提升至 500 MB，带宽从 80 TB/s 提升至 150 TB/s，新增 Fabric Expansion Logic (FEL) 支持通过 DDR 扩展内存。FEL 是一个 FPGA 芯片，通过高带宽 SerDes 连接 LPU 阵列，可以将 KV Cache 等大数据结构卸载到 DDR 内存，同时保持与片上 SRAM 类似的访问延迟特性。

2.4 各功能单元深入分析

基于 ISCA 2020/2022 论文 [1][2]、HotChips 2024 演讲 [9] 和 OneFlow 技术文章 [7] 的深入分析，本节详细阐述 TSP 各功能单元的微架构设计和工作原理。芯片内还额外设置了 **1 个备用的 SuperLane**，用于在制造阶段容忍单个 SuperLane 的硬故障，通过硬件隐式替换实现对编译器透明的良率修复——这也是整颗芯片上最显著的一处不由编译器显式控制的冗余资源。

2.4.1 指令控制单元 (ICU) 与指令流调度

ICU 负责从指令存储器取指、译码，并将微指令垂直 (Northward) 分发至各功能切片。ICU 提供精确的时间控制机制——通过 REPEAT 指令实现指令的周期级重复，通

过 SYNC/NOTIFY 指令实现跨切片的屏障同步。编译器利用这些机制精确控制指令发射时机，确保指令与数据流在时空上精确对齐。ICU 不执行动态调度，仅按编译器预设的序列发射指令，这种简化使 ICU 和所有指令调度逻辑仅占芯片面积的不到 3%[9]。相比之下，传统 CPU 中用于分支预测、乱序执行、推测执行的控制逻辑通常占用 15%-30% 的芯片面积。这种极端简化的控制平面是 TSP 高计算密度的关键来源。

ICU 支持的通用指令包括：IFETCH(从 SRAM 获取指令到 ICU 开始执行)、NOP(空操作，可重复 N 个周期用于时序对齐)、SYNC(将 ICU 置于等待状态，直到收到 NOTIFY 通知)、NOTIFY(唤醒其他 ICU 实现锁步执行)。这种设计使编译器能够精确推理每条指令的执行时间。

2.4.2 矩阵运算单元 (MXM)

MXM 是 TSP 架构的主力计算单元，每个 SuperLane 包含一个 320×16 的 MACC 阵列，整个芯片包含 4 个独立的 320×320 矩阵引擎。每个 MXM 平面存储 102,400 个权重，总计 409,600 个乘加器 (MACC)。在 900MHz 频率下，MXM 可提供约 750 TOPS(Int8Acc32) 的峰值算力，计算密度超过每平方毫米 1 TeraOps[9]。

MXM 通过 LOAD WEIGHTS(LW) 和 INSTALL WEIGHTS(IW) 指令从流寄存器加载权重，通过 ACTIVATION BUFFER CONTROL(ABC) 指令协调激活值的到达时机，通过 ACCUMULATE(ACC) 指令输出结果。整个计算过程的时间由编译器精确规划：权重安装、激活值流入、乘加运算、结果累加各阶段的时序在编译时确定，MXM 按固定周期完成操作。MXM 支持的数据类型和运算模式如表2所示。

表 2: MXM 矩阵运算单元支持的数据类型与运算模式

数值模式	最大尺寸	每 MXM 密度	结果类型
INT8	$[N, 320] \times [320, 320]$	2 个/平面	INT32
FLOAT16	$[N, 320] \times [160, 320]$	1 个/平面	FLOAT32

MXM 执行采用全流水线设计，每个周期可以接受新的输入，实现 sustained throughput。

2.4.3 向量运算单元 (VXM) 与逐元素操作

VXM 位于芯片的东西半球之间，包含 16 个处理元素 (PE) per lane，总计 5,120 个 32 位 ALU。每个 PE 每周期可执行一次 32 位运算或四次 INT8 运算。VXM 负责执行元素级运算，包括激活函数 (ReLU、GELU、Tanh)、归一化、点积等深度学习常见的逐点操作。

VXM 的数据通路包含 4×4 的 ALU 阵列。VXM 从流寄存器读取操作数，计算结果写回流寄存器——整个过程在固定周期内完成，编译器据此规划下游操作的启动时

机。VXM 支持的指令类别包括：一元运算 (如 mask、negate)、二元运算 (如 add、mul、sub)、类型转换 (定点与浮点互转)、超越函数 (ReLU、TanH、Exp、RSqrt)。

2.4.4 数据重排单元 (SXM)

SXM 是 TSP 的”瑞士军刀”，负责数据重塑、片上数据搬运和片外 I/O 操作。其核心功能包括：Shift up/down(在 lane 间上下移动数据 N 个位置)、Permute(320 输入到 320 输出的双射置换)、Distribute(在 SuperLane 内重排或复制数据)、Rotate(生成所有可能的旋转，支持 $n = 3$ 或 $n = 4$)、Transpose(16x16 矩阵转置)。

SXM 与 MEM 协同构成片上网络的 X-Y 维度：MEM 提供 X 维度 (东西向) 的数据流动，SXM 提供 Y 维度 (南北向) 的 lane 间数据交换。编译器在规划数据布局时，需同时考虑 MEM 的地址分配和 SXM 的置换模式，确保数据在正确的时间到达正确的 lane。在传统 GPU 中，卷积的 im2col 转换、注意力机制的 transpose 操作、多播通信等通常需要额外的内存访问，而 TSP 通过 SXM 在流寄存器之间直接完成，无需 SRAM 中转。

2.4.5 芯片间通信单元 (C2C) 与全局扩展

C2C 模块提供 Send/Receive 原语，支持跨芯片的 320 字节向量传输。Deskew 指令用于补偿芯片间的时钟漂移。编译器在生成多芯片程序的指令序列时，需将 C2C 操作纳入全局调度：发送操作在源芯片的指定周期执行，接收操作在目的芯片的对应周期执行，两者通过 HAC 计数器对齐。

Abts 2020 论文指出，芯片物理上集成了 16 个 C2C 模块共 64 条通道；但在实际部署的 TSP 系统中，每个芯片仅启用 11 条链路（共 44 通道），其中 7 条用于节点内全连接，4 条用于跨节点全局连接，总离线带宽达 2.64 Tb/s。

2.4.6 流寄存器机制与时空确定性

流寄存器 (Stream Register) 是 TSP 功能切片架构的核心创新。功能单元之间的数据通过流寄存器传递，形成”装配线”式的数据流。关键特性包括：单周期延迟 (相邻功能单元间数据传输仅需 1 周期)、无仲裁 (编译器静态分配流寄存器，无需硬件仲裁)、确定性时序 (编译器精确计算数据到达时间)。

数据在芯片上东西向流动，充分利用空间局部性。编译器通过简单的减法即可计算数据移动时间：$T_{travel} = N_{hops} \times T_{cycle}$。这种确定性使编译器能够精确编排指令执行，无需硬件干预。

每个 SuperLane 包含 16 个并行 lane，每个 lane 处理 1 字节数据。20 个 SuperLane 纵向排列，每条各贡献 16 字节，聚合形成最大 320 字节的向量处理宽度。Abts et al.

给出的流寄存器总带宽计算公式为：

$$B = 2\,\text{directions} \times 32\,\frac{\text{bytes}}{\text{lane}} \times 320\,\text{lanes} \times f \tag{1}$$

其中 32 bytes/lane 对应 32 条并发的数据流，320 lanes 为最大向量宽度，f 为时钟频率。在 1 GHz 主频下，流寄存器提供的总读写带宽为 **20 TB/s**，这一数字既是功能切片间数据流动的峰值速率，也构成了 SRAM 带宽（55 TB/s）设计的下限约束——SRAM 带宽必须高于流寄存器带宽，才能避免存储子系统成为数据流的瓶颈。

3 软件定义硬件与编译器架构

3.1 静态-动态接口划分

TSP 架构的核心理念是静态-动态接口划分。这一理念将计算系统的控制平面分为两部分：静态部分在编译时确定，包括指令调度、内存分配、数据布局、网络路由；动态部分在运行时确定，包括数据值、分支方向 (由编译器生成所有分支路径)、异常处理。传统架构将大部分控制逻辑放在硬件中动态执行，而 TSP 将静态部分完全移至编译器。这种划分的优势在于编译器拥有全局视角可以做出更优的调度决策，硬件简化节省的晶体管可用于更多计算单元，执行完全可预测便于验证和调试。

在 TSP 架构中，编译器扮演着核心角色：它知道每条指令将在哪个周期执行，知道每个数据位于哪个 SRAM 地址，知道每次芯片间通信的确切延迟，知道每个功能单元的状态。这种全知视角使编译器能够进行激进的优化：指令级并行、内存访问流水线、数据预取、循环展开等。更重要的是，这些优化在编译时验证，运行时无需任何猜测或回滚。

3.2 Groq 编译器架构

Groq 编译器 (groqit) 采用多阶段编译流程 [3]：前端接收 PyTorch/TensorFlow 模型并转换为中间表示 (IR)；图优化阶段执行算子融合、常量折叠、死代码消除；内存规划阶段为每个张量分配 SRAM 位置并规划生命周期；指令调度阶段生成 VLIW 指令序列并优化指令级并行；网络规划阶段生成芯片间通信的 SSN 路由表；后端生成 TSP 可执行的二进制代码。

内存分配是编译器面临的核心挑战之一。TSP 的 220 MB 或 500 MB SRAM 需要容纳：

- 模型权重（对于大模型可能需要分片存储在多个芯片）
- 激活值（中间计算结果）
- KV Cache（对于 LLM 推理）

编译器采用**内存池**(memory pool)和**张量生命周期分析**来优化内存使用,如代码2所示。

```
def allocate_memory(tensors, max_size):
    # 按生命周期排序张量
    sorted_tensors = sort_by_lifetime(tensors)

    memory_map = {}
    for tensor in sorted_tensors:
        # 寻找合适的空闲块
        slot = find_free_slot(
            memory_map, tensor.size, tensor.lifetime)
        if slot is None:
            # 尝试与已分配的张量共享内存
            slot = find_sharing_opportunity(
                memory_map, tensor)
        if slot is None:
            raise OutOfMemoryError(
                f"Cannot allocate {tensor.name}")
        memory_map[tensor] = slot

    return memory_map
```

Listing 2: 内存分配伪代码

TSP 编译器的调度算法需要确保:

1. 每个功能单元在同一时间只执行一个操作
2. 数据依赖被正确满足(生产者先于消费者)
3. 内存访问不冲突(同一 SRAM bank 同时只服务一个请求)
4. 网络链路不冲突(软件调度网络的约束)

这本质上是**资源约束调度**问题,编译器使用启发式算法(如列表调度)结合整数线性规划(ILP)来求解最优调度。

3.3 编译器优化技术详解

TSP 编译器的核心挑战是在满足资源约束的前提下,生成周期精确的 VLIW 指令序列。基于近期 VLIW 编译器优化研究 [13][14] 和 Groq 技术资料 [9],本节深入分析 TSP 编译器采用的关键优化技术。

3.3.1 周期精确指令调度

TSP 编译器执行**周期精确指令调度** (Cycle-Accurate Instruction Scheduling),这是其实现确定性执行的基础。由于功能单元以锁步方式执行,编译器需要:确定每条指令

的发出周期、计算操作数到达功能单元的精确时间、安排结果写回流寄存器的时序、处理功能单元间的流水线延迟。

这种调度可形式化为**资源约束的列表调度**问题：给定数据流图 $G = (V, E)$，其中节点 $v \in V$ 表示操作，边 $(u, v) \in E$ 表示数据依赖；给定资源约束 $R = \{r_1, r_2, ..., r_k\}$，每个资源有容量限制；目标是最小化调度长度 (makespan)，同时满足所有依赖和资源约束。

TSP 采用启发式列表调度算法：计算每个节点的优先级 (通常基于关键路径长度)，按优先级排序形成就绪列表，依次调度就绪节点到最早可用时隙，更新资源占用状态，重复直至所有节点调度完成。算法伪代码如代码3所示。

```
def list_schedule(dataflow_graph, resources):
    # 计算每个节点的关键路径长度作为优先级
    priorities = compute_critical_path(dataflow_graph)

    # 按优先级降序排序形成就绪列表
    ready_list = sort_by_priority(dataflow_graph, priorities)

    schedule = {}
    resource_state = initialize_resources(resources)

    while ready_list:
        node = ready_list.pop(0)

        # 计算最早开始时间(考虑数据依赖)
        earliest_time = compute_earliest_time(node, schedule)

        # 在最早时间后寻找满足资源约束的时隙
        for t in range(earliest_time, MAX_CYCLES):
            if check_resource_availability(node, t, resource_state):
                schedule[node] = t
                update_resources(node, t, resource_state)
                break

        # 将新就绪的节点加入列表
        ready_list.extend(get_newly_ready_nodes(dataflow_graph, node))
        ready_list.sort(key=lambda n: priorities[n])

    return schedule
```

Listing 3: 列表调度算法伪代码

3.3.2 模调度与软件流水线

对于循环密集型工作负载 (如 LLM 推理中的 token 生成)，TSP 编译器采用**模调度** (Modulo Scheduling)[14] 实现软件流水线。模调度的目标是找到最短的启动间隔 (Initiation Interval, II)，使得循环迭代可以流水线化执行而不发生资源冲突。

关键概念包括：启动间隔 II —连续两次迭代开始之间的时间；最小 II(II_{min}) —由资源约束或循环回边决定的理论下限；循环主体被划分为 II 个阶段，每个阶段形成一个**调度实体**。

模调度算法首先尝试 $II = II_{min}$，然后使用列表调度尝试将循环操作打包到 II 个阶段中。如果失败，则递增 II 重试。对于 TSP 的 VLIW 架构，成功的模调度可以实现每个周期启动一个新迭代，达到峰值吞吐量。

3.3.3 寄存器压力感知调度

传统列表调度仅关注指令级并行，可能产生大量同时活跃的值，导致寄存器溢出。TSP 编译器采用**寄存器压力感知调度** [15]，在并行性和寄存器使用之间取得平衡。

核心策略包括：在列表调度的优先级计算中引入寄存器压力估计；当寄存器压力接近阈值时，优先调度能释放寄存器的节点 (消费者)；使用图神经网络 (GNN) 预测调度决策对寄存器压力的影响；采用双寄存器技术 (Double Register Technique) 重叠寄存器加载与 MAC 运算。

研究表明 [15]，寄存器压力感知调度可以将因寄存器压力导致的不可调度程序比例从 18.6% 降至 0.8%，对 97.5% 的程序达到最优调度。

3.3.4 内存分配与数据布局优化

TSP 编译器的内存优化不仅包括简单的分配，还涉及复杂的数据布局决策：张量切片 (Tensor Tiling) —将大张量划分为适合 SRAM 的小块；bank 冲突避免—通过地址映射确保并行访问不同 bank；权重预加载—在计算当前层时预加载下一层权重；激活值复用—最大化相邻层间的激活值重用。

这些优化可形式化为**二维约束动态规划**问题 [13]：在时间和空间两个维度上优化张量放置。相比传统启发式算法，该方法可达到接近最优的解，效率提升 58.34%。

3.3.5 两阶段调度策略

TSP 编译器采用 **Pre-RA** 和 **Post-RA** 两阶段调度 [13]：

1. **Pre-RA 调度** (寄存器分配前)：基于虚拟寄存器进行粗粒度调度，确定指令大致顺序，估计寄存器需求。
2. **Post-RA 调度** (寄存器分配后)：基于物理寄存器进行精调，插入 load/store 操作，优化内存访问流水线。

研究表明 [13]，Post-RA 调度相比仅 Pre-RA 调度平均可提供 14.04% 的加速。这是因为寄存器分配后编译器知道确切的 spills 位置，可以更好地隐藏内存延迟。

3.4 运行时系统

与传统 GPU 需要复杂的运行时管理内核启动、内存分配、流同步不同，TSP 的运行时极为轻量：无需动态内存分配 (编译时已确定)，无需内核启动开销 (指令流连续执行)，无需流同步 (编译时已确保同步点)，无需动态调度 (硬件按序执行)。TSP 运行时主要负责加载编译后的程序到芯片、初始化 HAC/SAC 计数器、触发程序执行、处理异常和错误 (通过软件重放机制)。

TSP 架构通过多种机制确保可靠性 [2]：所有 SRAM 和网络链路都使用 ECC 保护，SerDes 链路使用前向纠错，检测到的错误触发软件重放重新执行出错的指令序列。由于执行的完全确定性，软件重放可以精确复现出错时的状态，大大简化了错误诊断和恢复。

4 多芯片同步与软件调度网络

4.1 从单芯片到多芯片的确定性扩展

单芯片的确定性执行相对容易实现——所有元件共享同一个时钟域，编译器可以精确计算每条指令的周期。但当系统扩展到多个芯片时，挑战出现：不同芯片的晶振频率存在微小差异导致时钟漂移，芯片间链路的延迟可能因温度、电压变化而波动，传统的屏障同步会引入显著延迟。

Groq 的解决方案是构建一个**软件定义的分布式同步系统**，通过硬件对齐计数器 (HAC)、软件对齐计数器 (SAC) 和专门的同步指令集，将单芯片的确定性保证扩展到由数千个芯片组成的大规模系统。

硬件对齐计数器 (HAC) 机制。每个 TSP 芯片维护一个 8 位硬件对齐计数器 (HAC)，以固定周期（每 256 个时钟周期）自由运行并回绕。HAC 的核心功能是为系统提供一个**粗粒度的全局时间基准**，使所有芯片能够在无需显式时钟同步的情况下达成时间共识。

HAC 的对齐过程通过测量和补偿芯片间链路延迟实现。当两个 TSP 通过 C2C 链路连接时，它们执行以下延迟表征协议（算法 1）：

实验测量表明，在一个 8-TSP 节点内，7 条 C2C 链路的平均延迟约为 216-217 个时钟周期，标准差约 2.6-2.9 个周期。这种低抖动特性使得 HAC 对齐可以达到周期级精度。

为扩展 HAC 协议到多跳网络，系统建立一棵以主 TSP 为根的**生成树**，每条边维护父子 HAC 关系。通过这种层次化对齐，整个网络收敛到共同的全局时间参考。

Algorithm 1 HAC 链路延迟表征与对齐

// 延迟测量阶段（在 Master 芯片 T_0 上执行）
for $i = 1$ **to** N_{iter} **do**
    $hac_{local} \leftarrow$ 读取当前 HAC 值
    通过 C2C 链路发送 hac_{local} 到 Slave 芯片 T_1
    // T_1 立即将接收值反射回 T_0
    $hac_{reflected} \leftarrow$ 接收反射值
    $RTT \leftarrow (hac_{reflected} - hac_{local}) \bmod 256$
    记录 RTT 到统计集合
end for
$L \leftarrow mean(RTT_collection)/2$ ▷ 估算单向延迟
$\sigma \leftarrow std(RTT_collection)$ ▷ 评估抖动
// 建立父子关系，T_0 作为 T_1 的 HAC 参考源
配置 T_1 周期性接收 T_0 的 HAC 值
当 HAC_1 收到 $HAC_0 + L$ 时，计算偏差 $\delta = HAC_1 - (HAC_0 + L)$
以配置速率调整 HAC_1 以减小 δ
// 约 256 个周期后，两计数器收敛到抖动范围内

初始程序对齐：启动同步。HAC 提供了全局时间基准，但程序启动需要更精确的同步机制。TSP 使用 DESKEW、TRANSMIT 和 RECV 指令实现跨芯片的程序对齐（图 7）。

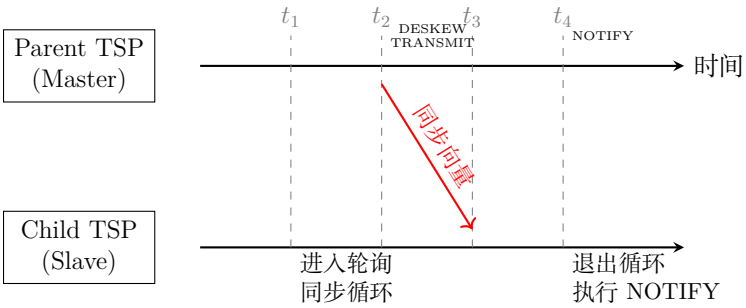

图 7: TSP 多芯片初始同步流程

同步流程的详细步骤如下：

1. **Child 进入轮询**（时间 t_1）：子芯片执行轮询同步循环，在每个 HAC 回绕边界检查是否收到来自父芯片的同步向量。

2. **Parent 发送同步**（时间 t_2）：父芯片执行 DESKEW 指令，暂停直到下一个 HAC 回绕边界，然后执行 TRANSMIT 发送同步向量。

3. **向量到达**（时间 t_3）：同步向量经过链路延迟 L 后到达子芯片。

4. **同时启动**（时间 t_4）：子芯片在下一个 HAC 回绕边界退出轮询循环，父芯片也在同一边界执行 NOTIFY。两个芯片在精确相同的时钟周期开始执行各自的程序。

DESKEW 指令的关键作用是将指令执行对齐到 HAC 回绕边界，伪代码如下：

```
void DESKEW() {
    stall_until_next_hac_epoch();
    resume_execution();
}
```

Listing 4: DESKEW 指令的语义

对于高度为 h 的生成树，总同步开销为 $(\lfloor L_{max}/period \rfloor + 1) \times h$ 个周期，其中 L_{max} 是树中最长边的延迟。这一开销仅在程序启动时发生。

运行时去偏：时钟漂移补偿。HAC 对齐建立了共同的启动时间，但长时间运行（如大模型推理的数百层计算）中，独立晶振的频率差异会导致累积漂移。TSP 通过**软件对齐计数器 (SAC)** 和 RUNTIME_DESKEW 指令解决这一问题。

SAC 是与 HAC 同周期的自由运行计数器，但**不**与上游 HAC 同步。HAC 与 SAC 的差值 $\delta_t = HAC - SAC$ 表示本地时间与全局时间的漂移量。RUNTIME_DESKEW 指令利用这一差值进行动态补偿：首先读取 HAC 与 SAC 的差值 δ（$\delta > 0$ 表示本地快于全局，需多等待；$\delta < 0$ 表示本地慢于全局，需少等待），然后将目标停滞周期与 δ 相加得到实际停滞周期并执行。其伪代码如下：

```
void RUNTIME_DESKEW(int target_stall_cycles) {
    int delta = read_hac() - read_sac();
    int actual_stall = target_stall_cycles + delta;
    stall(actual_stall);
}
```

Listing 5: RUNTIME_DESKEW 指令的语义

编译器在程序关键点（如 Transformer 层边界、大规模 All-Reduce 操作前）插入 RUNTIME_DESKEW，确保长期运行的稳定性。实验表明，BERT-Large 在 4 个 TSP 上运行 24,240 次的延迟分布显示 99% 的推理延迟低于 1225μs，所有运行都在 1300μs 内完成，验证了同步机制的有效性。

4.2 软件调度网络 (SSN)

传统高性能互连网络（如 InfiniBand、NVLink）采用动态路由和硬件仲裁，导致延迟不可预测。TSP 的**软件调度网络 (SSN)** 将所有路由决策前移至编译时，运行时网络仅执行预先计算的调度。

SSN 核心设计原则包括：

- **流量模式先验已知**：ML 模型的静态计算图在编译时完全确定，所有芯片间通信的时间、源、目的地均可预测。
- **调度而非路由**：传统网络中数据包在运行时通过查找表确定下一跳；SSN 中编译器精确编排每个向量在每跳链路上的发射时间。

- **确定性负载均衡**：利用 Dragonfly 拓扑的路径多样性，编译器可将大张量的向量分散到多条路径，避免拥塞而无需硬件自适应路由。
- **无硬件仲裁与流控**：SSN 禁止硬件背压和动态仲裁，所有资源冲突在编译时解决。
- **前向纠错替代重传**：链路层使用 FEC 纠正传输错误，避免链路层重传引入的非确定性。

物理层实现。TSP 芯片间互连采用低摆幅差分信号，每通道 30 Gbps，通过 34 AWG 铜缆传输。节点内电缆长度 < 0.75m，机架内 < 2m，使用 QSFP 电接口降低成本。

节点内互联：全连接拓扑与线缆优化。单机箱（4U 节点）包含 8 颗 TSP，节点内部采用全连接（fully-connected）拓扑，需要 **28 根内部 C2C 线缆**实现节点内全互联。这一设计充分利用了机箱内部的布线密度优势——论文指出，约 73% 的线缆（44/60）可以使用短距离（< 0.75m）的低成本电信号线缆，仅机架间长距离连接需要使用昂贵的有源光缆。每颗 TSP 的 7 条本地链路精确对应全连接拓扑的需求：每颗芯片需要与节点内其他 7 颗芯片直接相连。这种设计的优势在于任意两颗 TSP 间的通信仅需 1 跳，延迟最低且完全确定。

Dragonfly 拓扑：从本地组到全局系统。跨节点互联采用 Dragonfly 拓扑，这是实现大规模扩展的关键。TSP 系统的 Dragonfly 实现具有四级封装层次（图 8）：

1. **芯片级**：单颗 TSP 在系统层面启用 11 条 C2C 链路（7 条本地 +4 条全局），本地链路专用于节点内通信，全局链路用于跨节点扩展。
2. **节点级**：8 颗 TSP 通过 28 根线缆形成全连接本地组。所有全局链路聚合形成 32 端口虚拟路由器（8 TSP $\times$ 4 全局链路），作为 Dragonfly 的基本构建单元。
3. **机架级**：9 个节点（72 颗 TSP）组成一个机架。机架内部通过全局链路实现节点间互联，形成 288 个全局端口（9 节点 $\times$ 32 端口）。其中 144 个端口用于机架内部双倍连接（满足 2$\times$ 内部加速比要求），剩余 144 个端口用于跨机架连接。
4. **系统级**：最多 145 个机架组成完整系统，总计 10,440 颗 TSP。任意两颗 TSP 间最多 5 跳（源机架内 2 跳 + 全局 1 跳 + 目标机架内 2 跳），端到端延迟小于 3 微秒。

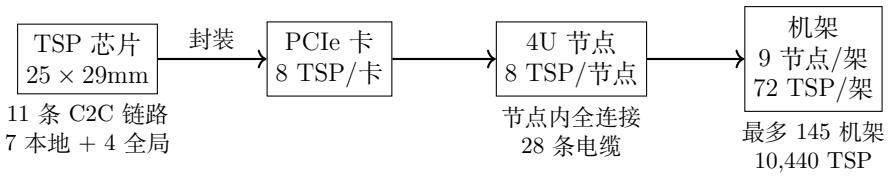

图 8: Groq TSP 系统的 4 级封装层次。从芯片级的 11 条 C2C 链路（7 本地 +4 全局），到节点级的 28 条线缆全连接，再到机架级的 288 端口虚拟路由器，最终扩展到系统级的 10,440 颗 TSP，整个架构呈现出清晰的层次化带宽特性。

路由与流控：编译时调度的网络抽象。SSN 的路由决策完全在编译时完成，运行时网络仅执行预设的调度。Dragonfly 拓扑的直径特性使得任意两个 TSP 间**最多 5 跳**（大规模系统中为源机架内 2 跳 + 全局 1 跳 + 目标机架内 2 跳）；而当系统**在 33 个节点（264 颗 TSP）以内全连接扩展**时，任意节点之间均有直接全局链路，直径**仅为 3 跳**（源节点内 1 跳 + 全局 1 跳 + 目标节点内 1 跳）。

SSN 采用类似 **Wormhole Switching** 的流控机制：320 字节向量作为流控单元 (flit) 直接穿过网络，交换节点在收到包头后即可开始转发，无需等待整个 flit 完整接收。这种设计与 SSN 的确定性调度哲学一致——编译器预先计算每个向量的端到端延迟，运行时无需动态仲裁或背压。错误恢复采用**前向纠错 (FEC)** 机制（而非链路层重传），避免了重传引入的非确定性延迟。

在传统网络中，当多个 TSP 同时向同一目标发送数据时，会在共享链路上产生竞争，导致拥塞和背压。SSN 采用不同的方法：编译器在编译时即知晓完整的流量模式，可以**时间复用**冲突链路（将冲突流量调度到不同时隙），或**空间分散**流量（通过非最小路径路由）。这种软件调度保证了链路的确定性使用，彻底消除了拥塞和背压。

通信协议。SSN 的通信协议与其极简设计理念一致。**数据包格式**体现了这一哲学：每个传输单元为固定 **328 字节**（含 320 字节有效载荷 +8 字节控制开销），编码效率达 97.5%。不同于传统网络需要携带目的地址、路由标签等元数据，TSP 数据包仅包含：5 字节头部（含类型标识、同步位、起始标记）、320 字节纯数据体、以及 3 字节尾部（含结束标记和空闲填充）。

软件通信模型。SSN 采用独特的**推送式通信模型**。路由信息已编码在编译器生成的指令流中——发送方在精确周期执行 SEND 指令，接收方同步执行 RECV 指令，无需运行时解析包头。传统 RDMA 采用请求-响应模式（发送 Read 请求 → 等待响应），而 SSN 中编译器预知数据产生和消费时间，主动将数据推送到消费者，消除请求往返延迟。这种”计算与通信协同调度”的模式使 All-Reduce 等集合通信能够在数微秒内完成，BERT-Large 规模的 All-Reduce 在 8 个 TSP 上可达到接近理论峰值的带宽利用率。

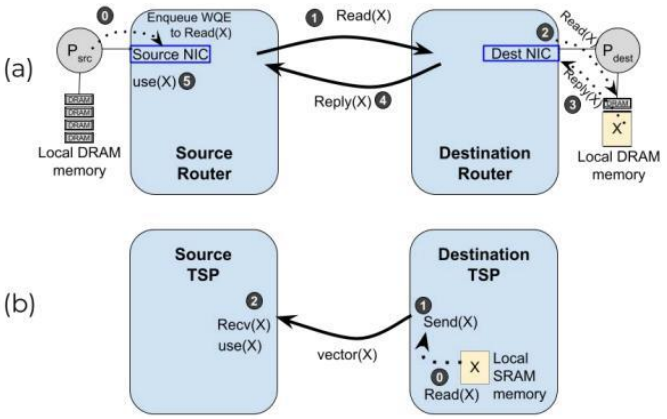

图 9: 传统网络 RDMA 通信模型 (a) 与 TSP 推送式通信模型 (b) 的对比。传统模型需要请求-响应往返，而 TSP 模型通过编译器预知数据产生时间直接推送，消除请求延迟 [2]

表 3对比了 SSN 与传统网络的特性：

表 3: 软件调度网络与传统网络对比

特性	传统网络 (InfiniBand/NVLink)	SSN
路由决策	运行时动态路由	编译时静态调度
仲裁机制	硬件仲裁器	无（调度避免冲突）
拥塞控制	背压、自适应路由	编译时避免
延迟预测	统计性，有尾延迟	确定性，无抖动
通信模型	请求-响应	推送式
队列/缓冲	深缓冲（KB 级）	无（Wormhole-like Cut-Through）
虚拟通道	需要（避免死锁）	不需要
错误恢复	链路层重传（非确定性）	FEC 前向纠错（确定性）
包头开销	5-10%	2.5%（328 字节/320 字节）

SSN 的确定性特性使得 BERT-Large 在 4-TSP 系统上的 24,240 次推理中，编译器预测的延迟与实际测量值偏差在 2% 以内。这种可预测性对于实时推理和分布式训练中的流水线并行至关重要。

5 与 GPGPU 架构的对比分析

5.1 计算密度与能效

表4对比了 TSP 与同期 GPU 的关键指标。图10进一步从架构设计哲学层面展示了 TSP 与传统 CPU/GPU 的根本差异。

从表中可以观察到，TSP 的计算密度 (Ops/sec/transistor) 是 V100 的 5 倍 [1]。这种优势主要来源于：晶体管用于计算而非控制逻辑 (无缓存、无乱序执行单元)，极高的片上内存带宽 (80 TB/s vs 2 TB/s)，以及确定性执行消除了性能抖动。

表 4: TSP 与 GPGPU 架构对比

指标	TSP (2020)	V100	A100
工艺节点	14nm	12nm	7nm
芯片面积	725 mm^2	815 mm^2	826 mm^2
晶体管数量	26.8B	21.1B	54.2B
峰值性能 (FP16)	820 TOPS	130 TFLOPS	312 TFLOPS
SRAM/缓存容量	220 MB	16 MB	40 MB
内存带宽	80 TB/s (SRAM)	900 GB/s (HBM2)	2 TB/s (HBM2e)
计算密度 (每晶体管性能)	30.6K	6.2K	5.8K
ResNet50 Batch=1	20.4K IPS	4-5K IPS	5-6K IPS

特性 (Feature)	**Groq TSP (张量流处理器)**	**传统架构 (CPU/GPU)**
调度方式	静态调度: 所有操作由编译器在编译时预先规划好.	动态调度: 由硬件调度器在运行时动态决定指令和数据流.
核心组件	仅包含执行单元和内存, 无动态控制逻辑. 硬件执行一个预先确定的"脚本".	包含复杂的反应式组件, 如缓存, 乱序执行引擎, 硬件调度器, 仲裁器等.
性能可变性	零方差: 每次运行的性能完全相同.	高方差: 性能受缓存命中率, 分支预测, "邻居"干扰等因素影响, 存在抖动.
长尾延迟	不存在: 架构设计从根本上消除了计算核心的长尾延迟.	普遍存在: 动态调度和资源争用是长尾延迟的主要来源.
编译器角色	核心决策者: 负责全部的执行规划, 资源分配和时间编排.	辅助角色: 主要负责将高级语言翻译成机器指令, 大量优化依赖于硬件在运行时动态完成.
调试与分析	简单: 静态性能报告在编译时即可生成, 无需在硬件上运行. 行为可重复, 易于调试.	复杂: 需要动态分析 (profiling) 工具来理解性能瓶颈, 结果可能因运行环境而异.

图 10: TSP 与传统架构 (CPU/GPU) 的核心差异对比 [3]。涵盖调度方式、核心组件、性能可变性、长尾延迟、编译器角色、调试与分析等六个维度。

5.2 延迟与可预测性

对于 LLM 推理等延迟敏感型应用，TSP 表现出显著优势 [8]。早期 ResNet50 图像分类测试显示，TSP 在 batch size=1 时达到 **20.4K IPS**，比同期 GPU 和 TPU 快 4-5 倍。进入 LLM 时代后，性能分析表明，在 LLM 推理场景下，Groq LPU（576 芯片配置）对比 DGX H100（8 卡）呈现显著差异：在延迟优化场景（batch size=3）下，Groq 的 per-user 吞吐约为 200-300 token/s，而 H100 仅能达到约 149 token/s；更关键的是系统总吞吐——Groq 可达约 55K token/s，而 H100 仅约 449 token/s，相差约两个数量级。要达到 Groq 的延迟水平，需要约 25 台 DGX（200 张 H100）做数据并行。这凸显了**确定性架构在小批量推理场景的效率优势**：通过编译时精确调度消除运行时开销，实现近 100% 的硬件利用率。

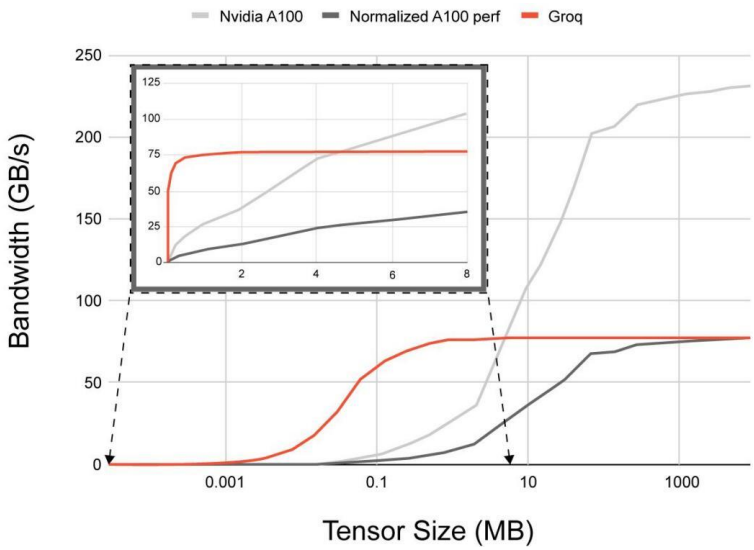

图 11: TSP 与 NVIDIA A100 的通信带宽随张量大小变化对比。TSP 凭借纯 SRAM 架构可在极小张量尺寸下立即达到接近峰值带宽（约 80 GB/s 有效带宽），而 A100 需要数十 MB 级张量才能接近峰值，小批量场景下带宽利用率显著低于 TSP[1]

单 token 延迟方面，TSP 可实现亚微秒级延迟，而 GPU 通常为毫秒级。尾延迟方面，TSP 的 P99 延迟几乎等于平均延迟，GPU 的 P99 延迟可能是平均值的 2-3 倍。抖动方面，TSP 无抖动，GPU 因缓存未命中、调度竞争等因素产生显著抖动。确定性执行带来的不仅是性能优势，还包括：可以为关键任务提供硬实时保证，相同的输入总是产生相同的输出和时序从而简化调试，以及消除了时序侧信道攻击的可能性。

5.3 编程模型与生态

TSP 的软件定义方法带来了编程复杂度的权衡。优势在于编译器自动处理并行化、内存管理、同步，确定性执行简化了推理和调试，无需手动优化 (如 CUDA 中的共享内存管理、warp 调度)。劣势在于不支持动态形状，不支持动态内存分配 (如 paged attention)，需要重新编译模型以适配不同 batch size。

Groq 面临的最大挑战是生态系统 [3]：软件栈不成熟 (相比 CUDA 的成熟生态)，开源社区对 TSP 架构的了解和支持有限，专用硬件难以移植到其他平台存在供应商锁定风险。

6 NVIDIA 收购与技术整合展望

6.1 200 亿美元收购的技术逻辑

2024 年，NVIDIA 以 200 亿美元收购 Groq[6]。这一交易的技术逻辑在于：推理市场的竞争——随着 AI 推理需求爆发，低延迟、高吞吐的推理加速器变得至关重要；技术互补性——NVIDIA 的 GPU 在训练领域无敌，但在低延迟推理领域面临 Cerebras、SambaNova 等挑战；以及人才获取——Groq 团队拥有独特的确定性计算专业知识。

NVIDIA GPU 面临的根本问题是 [3]：内存墙 (HBM 带宽增长跟不上计算能力增长)，控制开销 (CUDA core 中约 30% 的晶体管用于控制逻辑)，以及不可预测性 (缓存、分支预测、动态调度导致执行时间难以预测)。

6.2 从 TSP 到 Groq 3 LPX：架构演进与产业定位的收敛

2025 年 12 月，NVIDIA 宣布以约 200 亿美元达成对 Groq 资产与技术授权的重大交易，Groq 创始人及核心工程团队加入 NVIDIA，并在随后的 GTC 上发布了基于第三代 Groq LPU 的 **LPX** 机柜系统。这标志着 Groq 的技术路线正式融入 NVIDIA Vera Rubin 平台生态。相比第一代 TSP（Abts 2020），Groq 3 在芯片规格和系统定位上均发生了深刻变化。

芯片级规格跃升。Groq 3 LPU 将片上 SRAM 从第一代的 220 MB 提升至 **500 MB**，SRAM 带宽从约 80 TB/s 提升至 **150 TB/s**，峰值算力达到 **1.2 PFLOPS (FP8)**。最大向量宽度从 320 bytes 扩展至约 768 bytes。与此同时，C2C 互连能力实现了数量级增长：从第一代系统启用的 11 条链路（44 通道）跃升至 **96 个 112G SerDes 接口**，单芯片 C2C 单向带宽达 **10 Tbps**，系统级规模从数百芯片的小集群扩展到单个 LPX 机柜即容纳 **256 颗 LPU**。

系统形态：LPX 机柜。一个 LPX 机柜由 32 个 ComputeTray 组成，每个 Tray 含 8 颗 LPU。官方标称整柜 AI 推理算力达 **315 PFLOPS**，SRAM 总容量 **128 GB**，Scale-up 带宽 **640 TB/s**。这一系统规模使 Groq 从” 单芯片优化” 走向了”rack-scale 确定性扩展”。

然而，最具战略意义的转变并非规格数字本身，而是**产业定位的根本转向**——Groq LPU 不再是试图与 GPU 竞争的 standalone 推理加速器，而是作为 NVIDIA Vera Rubin GPU 的异构协处理器。

Attention-FFN Disaggregation (AFD)。NVIDIA 官方将这一异构协作模式定义为 AFD：在 decode 阶段，**Rubin GPU** 负责需要大容量 HBM 和高算力的 full-context

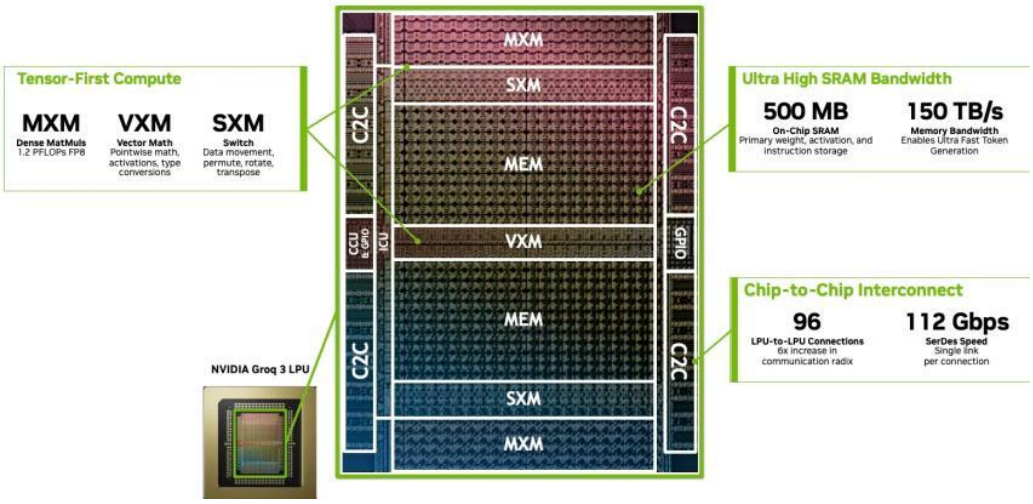

图 12: Groq 3 LPU 芯片架构与功能切片排布。芯片采用功能切片设计，包含 MXM（矩阵运算）、VXM（向量运算）、SXM（数据搬移）和 MEM（存储）等模块，片上 SRAM 容量 500 MB，带宽 150 TB/s。(来源: NVIDIA Groq 3 LPX 技术博客, 2025)[4]

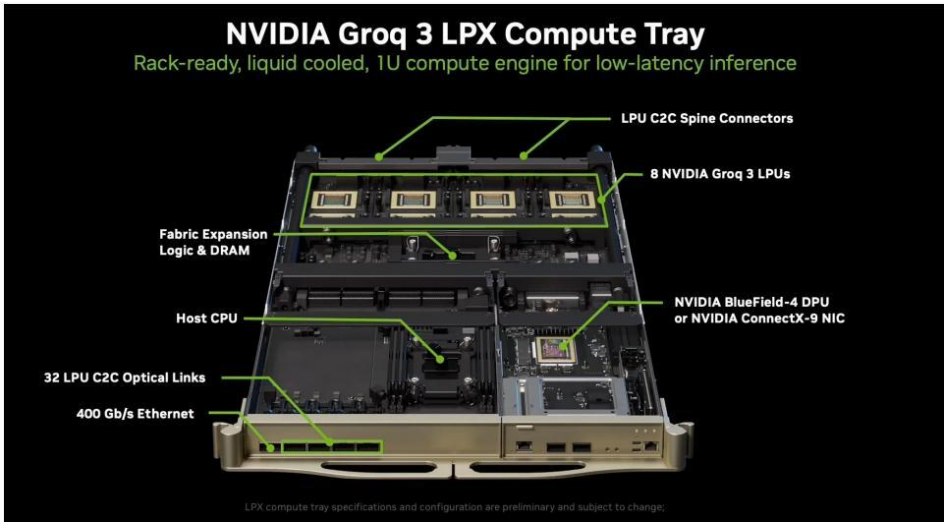

图 13: LPX Compute Tray 内部结构。每个 Tray 为 1U 液冷设计，含 8 颗 Groq 3 LPU、Fabric Expansion Logic (FEL) 与 256 GB DRAM、Host CPU、BlueField-4 DPU 以及 32 条 LPU C2C 光互连。(来源: NVIDIA Groq 3 LPX 技术博客, 2025)[4]

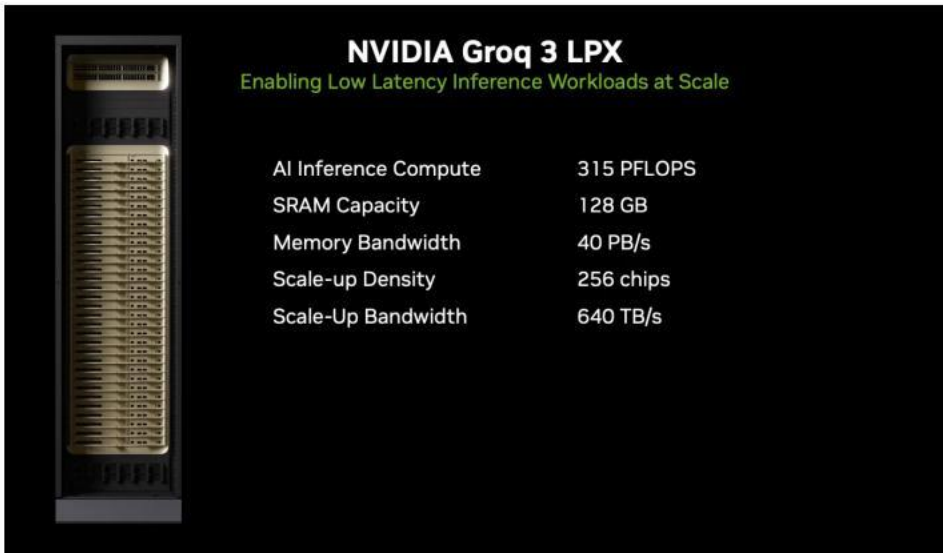

图 14: LPX 机柜系统。一个 LPX 机柜由 32 个 ComputeTray 组成，总共 256 颗 LPU 和 128 GB 聚合 SRAM，提供 315 PFLOPS AI 推理算力。(来源: NVIDIA Groq 3 LPX 技术博客, 2025)[4]

Attention（以及 Prefill 阶段），而 **Groq 3 LPU** 负责对带宽极度敏感、对小 batch 效率要求苛刻的 sparse MoE FFN。两个处理器每 token 交换中间激活，形成一个"two-engine decode loop"。这是数据流架构在产业实践中的一次关键收敛：它承认了纯 SRAM 设计在 >1T 参数 MoE 面前的容量瓶颈，同时通过异构分工保留了确定性执行在延迟敏感路径上的带宽优势。

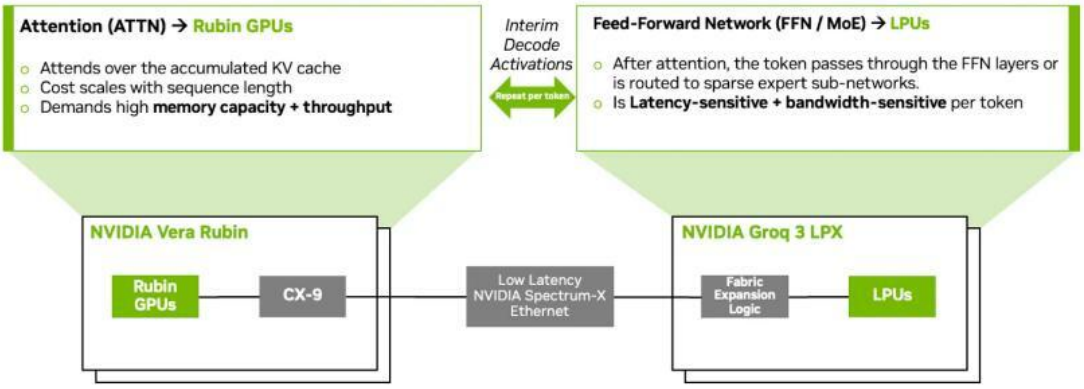

图 15: Attention-FFN Disaggregation (AFD) 异构解码架构示意。在 decode 阶段，Rubin GPU 负责 full-context Attention（需要大容量 HBM 和高吞吐），Groq 3 LPU 负责 sparse MoE FFN（对单 token 延迟和带宽极度敏感）；两个处理器每 token 交换中间激活，形成 two-engine decode loop。(来源: NVIDIA Groq 3 LPX 技术博客, 2025)[4]

容量瓶颈的妥协：Fabric Expansion Logic (FEL)。为支撑远超 128 GB SRAM 容量的大规模模型，LPX 系统引入了 **Fabric Expansion Logic**（当前为 time-to-market 的 FPGA 方案，带 256 GB DRAM）。FEL 作为确定性 LPU 域与非确定性外部网络之间的" 大坝"，缓存无法常驻 SRAM 的专家参数，并在层间计算时进行 Prefetch。这相当于在 Groq 原本的纯 SRAM 两级存储体系中，插入了一个外部容量层——标志着 Groq

从”彻底规避内存墙”的理想主义，走向了”用异构层次管理内存墙”的现实主义。

Speculative Decoding 与 NVIDIA Dynamo。Groq 3 LPX 的另一官方应用场景是跨异构处理器的 **speculative decoding**：LPX 以低延迟运行 EAGLE-3 等轻量草稿模型生成 draft tokens，Rubin GPU 则以高吞吐验证并 finalize。NVIDIA Dynamo 框架负责编排两种处理器之间的 token 流转。这进一步印证了 Groq LPU 在新生态中的角色——不是通用替代者，而是填补 GPU 在”小 batch、超低延迟、超高带宽”场景下效率死角的专用拼图。

NVIDIA 可能从以下几个方面整合 Groq 技术 [3]：将 Groq 的确定性调度技术引入 CUDA 编译器，在 GPU 中增加更多 SRAM 减少 HBM 访问，将 SSN 技术应用于 NVLink 和 InfiniBand，利用 Groq 的 SRAM 设计经验优化未来 3D 堆叠 GPU。技术整合也面临挑战：如何在保持 CUDA 兼容的同时引入确定性执行，大容量 SRAM 的成本远高于 HBM，以及 TSP 的专用设计是否适用于除 LLM 以外的 AI 工作负载。

从 TSP 的”软件定义硬件、纯 SRAM、独立芯片”到 Groq 3 LPX 的”异构 AFD、FPGA 容量扩展、机柜级确定性域”，这一演进揭示了数据流架构在产业落地时的真实轨迹：极致的静态调度优势需要通过与通用处理器的协同、以及对存储层次的务实妥协，才能在大模型时代维持商业价值。

7 结论与批判性思考

7.1 主要贡献总结

Groq TSP 架构对计算机体系结构领域做出了重要贡献。软件定义硬件范式证明了通过编译器完全控制硬件的可行性，将传统硬件控制逻辑移至软件层。确定性计算架构展示了消除所有非确定性元件后可以实现完全可预测的高性能执行。功能切片微架构提出了与核心-缓存模型根本不同的组织方式，实现了极高的数据局部性和带宽利用率。大规模同步机制 HAC/SAC 解决了分布式系统中时钟同步的经典难题。软件调度网络证明了编译时路由可以替代动态路由，消除网络拥塞和尾延迟。

7.2 局限性与批评

尽管 TSP 架构具有许多创新之处，但也存在明显的局限性。在适用域方面，TSP 的设计高度针对深度学习推理优化，对其他类型的工作负载支持有限：不适合训练 (需要大量权重更新，SRAM 容量不足)，不适合动态图 (control flow 必须在编译时确定)，不适合稀疏计算 (缺乏高效的稀疏矩阵支持)。

SRAM 容量瓶颈是另一个关键问题。220 MB 或 500 MB 的 SRAM 容量严重限制了可支持的模型规模 [5]：70B 参数的 LLM 需要 300-500 个芯片，576 卡集群的互联开销显著，无法使用 paged attention 等动态内存优化技术。

编译复杂度也不容忽视。确定性执行要求编译器解决 NP-hard 的调度问题，对于复杂模型，编译时间可能成为瓶颈。

7.3 未来研究方向

基于对 TSP 架构的分析，以下方向值得进一步研究。混合架构研究如何在 GPGPU 和 DSA 之间找到最佳平衡点，NVIDIA 的 Rubin+LPU 融合是一个尝试。动态确定性探索是否可以在保持确定性的同时支持有限的动态行为 (如动态 batching)。编译器-硬件协同设计进一步优化编译器算法以支持更大规模的模型和系统。可靠性机制方面，软件重放是确定性架构的独特优势，值得深入研究。建立确定性执行的行业标准有助于促进生态系统发展。

7.4 最终思考

Groq TSP 架构代表了一种根本不同的设计哲学。它挑战了”硬件必须足够通用以适应各种工作负载”的传统观念，证明了专用化在特定领域的巨大价值。200 亿美元的收购价格反映了业界对这一技术路线的认可。然而，技术成功不等于商业成功——TSP 能否在 NVIDIA 的框架内继续发展，能否在更广泛的 AI 工作负载中证明其价值，仍有待观察。

无论如何，TSP 架构为计算机体系结构社区提供了宝贵的思想实验：如果我们重新设计处理器，不受既有范式的约束，会做出什么不同的选择？这个问题本身就具有重要的启发意义。

参考文献

[1] Dennis Abts, Jonathan Ross, Jonathan Sparling, et al. Think Fast: A Tensor Streaming Processor (TSP) for Accelerating Deep Learning Workloads. In *Proceedings of the 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2020. DOI: 10.1109/isca45697.2020.00023

[2] Dennis Abts, Garrin Kimmell, Andrew Ling, et al. A Software-Defined Tensor Streaming Multiprocessor for Large-Scale Machine Learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, pp. 567-580, 2022. DOI: 10.1145/3470496.3527405

[3] zartbot. Groq LPU 架构详解. Technical Report, 2025. Available in knowledge base.

[4] zartbot. 详细谈谈 Rubin + Groq 3 LPU 架构. Technical Report, 2026. Available in knowledge base.

[5] ljw. Groq 多芯片同步方案. Technical Analysis, 2025. Available in knowledge base.

[6] 看到新闻的时候. 谈谈那个被 NV 看上值 20B 的 Groq. Technical Analysis, 2024. Available in knowledge base.

[7] OneFlow. Groq：从头设计一个张量流式处理器架构. Technical Article, 2025. Available in knowledge base.

[8] 知返. Groq LPU 架构和 LLM 推理性能分析. Technical Analysis, 2024. Available in knowledge base.

[9] Dennis Abts. The Groq Tensor Streaming Processor: Software-defined Scale-out Architecture for Deep Learning. In *HotChips 34*, 2024. Available in knowledge base.

[10] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*, 6th Edition. Morgan Kaufmann, 2019.

[11] Norman P. Jouppi, Doe Hyun Yoon, George Kurian, et al. Ten Lessons From Three Generations Shaped Google's TPUv4i. In *Proceedings of the 48th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2021.

[12] NVIDIA. NVIDIA H100 Tensor Core GPU Architecture. Technical Report, 2024.

[13] Yuzhou Deng, Mei Wen, Dong Wang, et al. Optimizing VLIW Instruction Scheduling via a Two-Dimensional Constrained Dynamic Programming. In *ACM Transactions on Design Automation of Electronic Systems*, 2024. DOI: 10.1145/3643135

[14] Mei Wen, Yuzhou Deng, Dong Wang, et al. 分簇式 VLIW 密码专用处理器的编译器后端优化研究. In *Journal of Chinese Computer Systems*, Vol. 44, No. 10, 2023.

[15] Sofia, et al. A Graph Neural Network Approach to Improve List Scheduling Heuristics Under Register-Pressure. In *IEEE International Conference on Intelligent Decision Technologies*, Sofia, Bulgaria, 2024.

[16] Ye et al. CHARM: Composing Heterogeneous Accelerators for Matrix Multiply on Versal ACAP Architecture. In *ACM Transactions on Reconfigurable Technology and Systems*, 2024.

[17] SPECTRUM Team. Time-Predictable Software-Defined Architecture with SDF-Based Compiler Flow for 5G Baseband Processing. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020.